MOSIP DEC{}DE · TECHNICAL REFERENCE

Inji Conformance Harness — API Documentation

MOSIP DEC{}DE submission reference · 2 November 2026

This document describes the APIs used by the existing harness, their request and response formats, authentication boundaries, example use cases, and known gaps. It is based on the implementation and retained release evidence. Examples are sanitized contract illustrations, not new successful executions.

Status: the harness is an implemented testing prototype. Complete end-to-end protocol acceptance remains incomplete. The latest fresh harness regression run recorded 90 passes; this does not establish OpenID conformance. Docker-dependent APIs were not revalidated during this documentation export.

- 1. Scope and implementation ownership
- 2. Addresses, routing and media types
- 3. Authentication, trust and sensitive data
  - 3.1 Suite API
  - 3.2 Certify
  - 3.3 Verify
  - 3.4 Storage and disclosure
- 4. OpenID suite REST API
  - 4.1 Create a test plan
  - 4.2 Read a plan
  - 4.3 Create a module runner
  - 4.4 Start and observe module execution
  - 4.5 Logs and plan export
- 5. Certify APIs and offer handoff
  - 5.1 Health and discovery
  - 5.2 Create pre-authorized credential data
  - 5.3 Retrieve an offer
  - 5.4 Native token exchange
  - 5.5 Credential issuance boundary
- 6. Verify APIs and DCQL presentation flow
  - 6.1 Create a backend VP request
  - 6.2 Relay actual authorization details to the suite
  - 6.3 Submit a presentation
  - 6.4 Poll request state
  - 6.5 Retrieve and verify results
  - 6.6 Browser-session APIs and unfinished UI integration
  - 6.7 Standalone native credential verification and status resolution
- 7. Example use cases
  - 7.1 Evaluate the API contract without a running stack
  - 7.2 Trusted local health request

    - 7.3 Create a real Verify request
    - 7.4 Run full diagnostic workflows
    - 7.5 Retrieve results using the Python HTTP contract
- 8. Timeouts, retries and error handling
- 9. Local output contract and build exits
- 10. Known limitations and current evidence
- 11. Source and documentation index

1. Scope and implementation ownership

The Python runner is an **API client and command-line tool**, not a new REST server. There is no harness-owned HTTP port and no invented `/run-conformance` endpoint. The Java TestNG bridge launches Python as a subprocess. REST endpoints in this reference belong to the OpenID Foundation suite, Inji Certify, or Inji Verify. Application behavior must be checked against the selected release.

Component	Responsibility	Main implementation
Suite REST client	Create plans/modules, start and poll execution, collect logs and exports	`runner/conformance_runner.py`
Certify handoff	Create native pre-authorized credential offer and pass its actual URI to suite	`runner/handoff_certify.py`
Verify handoff	Create real DCQL request, relay authorization details, retrieve verification result	`runner/handoff_verify.py`
Inji HTTP client	Separate trusted-TLS requests with deadlines and limited safe retries	`runner/common.py`
Configuration renderer	Load literal environment inputs and render typed JSON plans	`runner/render_plans.py`
Reporting	JSON/Markdown, per-plan evidence, TestNG and Extent	`runner/merge_results.py`, `testrig-bridge/`
Execution budgets	Evaluate configured component/module/combined limits	`runner/benchmarks.py`

Reference versions:

- Certify: `v1.0.0-alpha.1`, source commit `8fdf71656979a72fa8263766d4dec3cc25127b76`.
- Verify: `v1.0.0-alpha.1`, source commit `4c4289026968beb535d3dbdd500947152c43f708`.
- Later retained suite execution: `release-v5.3.4`, source commit `23353b36957d6466010de767c812d2c91ed37638`.
- Local derived images contain reviewed native metadata/status fixes. They must not be described as unmodified upstream images.

This is the harness integration surface, not an exhaustive catalogue of all upstream administration, issuance-plugin or account-management APIs.

2. Addresses, routing and media types

Purpose	Host-facing address	Advertised address inside the test deployment
Suite REST API	`https://localhost:8443`	`https://suite.test:8443`
Certify API	`https://localhost:8446/v1/certify`	`https://certify.test:8446/v1/certify`
Verify API	`https://localhost:8447/v1/verify`	`https://verify.test:8447/v1/verify`

The issuer identifier is origin-only, `https://certify.test:8446`. The gateway maps root well-known requests to Certify's servlet path. Do not append a servlet path to issuer metadata merely to make routing work. Host callers use localhost; suite/container callbacks use their configured public DNS names.

Common media types:

Operation	Request / response
Suite API and most Inji APIs	`application/json`
Native token exchange and Verify direct-post	Request `application/x-www-form-urlencoded`
Suite plan export	Response `application/zip`
Signed issuer metadata when actually supported	`application/jwt`
Verify signed authorization request, outside selected unsigned profile	`application/oauth-authz-req+jwt`
Native JWT Token Status List checker	`application/statuslist+jwt`

Strings such as `<plan-id>`, `<request-id>` and `<signed-token>` below are documentation placeholders. They are not working credentials or valid tokens. Capture actual identifiers from responses; URL-encode path and query values.

3. Authentication, trust and sensitive data

3.1 Suite API

`OIDC_SUITE_TOKEN`, when supplied, adds `Authorization: Bearer ...` to the suite session. The local dev-profile suite does not require that token. This does not establish production authentication policy. Never publish token values.

`OIDC_SUITE_INSECURE=false` is the normal setting. Python trusts the development CA through `REQUESTS_CA_BUNDLE=./local/tls/ca.crt`. An explicit insecure setting affects only the isolated suite session; it must not be treated as a normal deployment requirement or proof of TLS correctness.

The suite session refuses unapproved origins, does not follow redirects, and translates the configured public suite origin into the host-facing URL. Suite credentials must not be forwarded to Certify, Verify or arbitrary callback hosts.

3.2 Certify

The local fixture permits selected development endpoints, including discovery, pre-authorized-data and OAuth routes. These fixture permissions are not a production access-control guarantee. Credential issuance additionally uses the native access-token validation path.

The inspected non-local Certify filter requires a Bearer JWT and validates signature, issuer, audience, subject, `client_id`, issue time and expiry. The issuance service also requires a credential-mapped scope and valid holder proof. Do not disable those checks to demonstrate issuance.

No external AS or registered external clients are provisioned. Native IAR is not a conventional external authorization endpoint. Resource-side DPoP proof validation remains an implementation gap; a DPoP-bound token cannot safely be accepted as an unchecked Bearer token.

3.3 Verify

The selected development REST handoff does not attach an OAuth bearer credential to Verify. HTTPS trust is still required. Browser session APIs use actual service cookies and their configured security attributes. Request/transaction identifiers are correlation values, not a substitute for production access control.

The suite supplies signed synthetic presentation material. Native Verify validates the submitted presentation and credentials. Do not replace signed values, skip status checks, or rewrite results to satisfy a test.

3.4 Storage and disclosure

Private keys and environment files remain under ignored private paths. Raw plans, logs, ZIP exports, credentials, codes and cookies may contain sensitive material. Do not publish them without review. Public examples use synthetic claims and placeholder identifiers only. Reports should include sanitized outcomes rather than token contents. Status-list retrieval uses normal JVM TLS validation.

4. OpenID suite REST API

All paths in this section are relative to the suite API base. The driver expects the exact status shown; a different status becomes an API/infrastructure error. Response examples show only fields consumed by the harness, not complete schemas.

4.1 Create a test plan

POST `/api/plan` — expected HTTP 201

Query parameters:

Name	Value
`planName`	Exact available suite plan name
`variant`	URL-encoded JSON object, not an opaque variant string

Headers: `Content-Type: application/json`, `Accept: application/json`. The JSON body is **the plan's config object alone**. Do not send the enclosing `planName`, `variant`, `config` wrapper as the body.

```
{
  "alias": "inji-verify",
  "description": "Inji Verify diagnostic execution",
  "client": {
    "client_id": "redirect_uri:https://verify.test:8447/v1/verify/v2/vp-submission/direct-post"
  },
  "credential": {
    "signing_jwk": "<actual private test JWK object loaded securely from file>"
  }
}
```

This is an abbreviated schema illustration. The `signing_jwk` placeholder above must be replaced with the actual **object**, not a string, by the renderer; use `configs/verifier-plan.json.tpl` for the complete selected configuration.

Response subset:

```
{"id": "<plan-id>"}
```

Use case: start an isolated Certify or Verify diagnostic plan. The runner then fetches the plan's modules; a created plan without runnable modules is not success.

4.2 Read a plan

GET `/api/plan/{planId}` — expected HTTP 200

No body. Response subset:

```
{
  "id": "<plan-id>",
  "modules": [
    {"testModule": "<actual-suite-module-name>", "variant": {}}
  ]
}
```

The harness requires a nonempty module list and uses each module's real name and variant. A diagnostic `--module` filter must match an available module exactly.

4.3 Create a module runner

POST `/api/runner` — expected HTTP 201

Query: `test=<module-name>`, `plan=<plan-id>`, `variant=<URL-encoded module variant JSON>`. Send JSON Content-Type even though there is no request body. Response subset: `{"id":"<test-id>"}`.

Create-from-plan and plan creation are writes. Do not retry after an ambiguous lost response: the suite may already have created the object.

4.4 Start and observe module execution

Method and route	Purpose	Expected status
`POST /api/runner/{testId}`	Start a configured runner once	200
`GET /api/info/{testId}`	Read persisted lifecycle/result state	200
`GET /api/runner/{testId}`	Read live browser/exposed interaction state	200
`DELETE /api/runner/{testId}`	Cancel an abandoned/timed-out runner	200

Persisted state example, with optional result only when relevant:

```
{"status":"FINISHED", "result":"FAILED"}
```

`CONFIGURED` triggers one start request. `WAITING` requires the live runner endpoint. `FINISHED` carries the suite result; `INTERRUPTED` remains an interruption. Persisted info is not a substitute for live handoff data.

Abbreviated live state, supplied by the actual suite:

```
{
  "browser": {
    "uriInputRequests": [{"submitUrl":"<suite-provided interaction URL>"}],
    "uploadsRequired": 1
  },
  "exposed": {
    "credential_offer_endpoint": "<suite-provided offer interaction URL>"
  }
}
```

Fields vary by module and state. Do not construct callback URLs from this illustration. The runner reads them from the live response. Some interaction GETs change state and are explicitly not retried after a lost response.

4.5 Logs and plan export

GET `/api/log/{testId}` — expected HTTP 200, JSON log data.

GET `/api/plan/export/{planId}` — expected HTTP 200, with `Accept: application/zip`. The driver verifies that the response is a nonempty, valid ZIP before saving it. An HTTP-200 HTML page is not an acceptable export.

Logs are saved per plan and original test ID. They can contain sensitive test configuration. Evidence retrieval failure remains infrastructure failure even when some protocol modules completed.

5. Certify APIs and offer handoff

5.1 Health and discovery

Route	Purpose
`GET /v1/certify/actuator/health`	Health JSON; require `status=UP`, not merely HTTP 200
`GET /.well-known/openid-credential-issuer` at public issuer origin	Actual issuer metadata through gateway mapping
`GET /.well-known/oauth-authorization-server` at public issuer origin	Actual native AS metadata through gateway mapping

The reviewed unsigned-metadata patch returns unsigned JSON when signed metadata is unavailable, preserving actual metadata values. A signed-metadata test can be SKIPPED; that is not a protocol pass. The native metadata exposes `interactive_authorization_endpoint`; no conventional `authorization_endpoint` has been fabricated or renamed.

5.2 Create pre-authorized credential data

POST `/v1/certify/pre-authorized-data` — expected HTTP 200, JSON.

Request field	Type and constraint
`credential_configuration_id`	Required nonblank identifier registered in Certify
`claims`	Required object; identity values must match the selected template/plugin
`expires_in`	Optional integer, 60 to 86,400 seconds; harness sends 600
`tx_code`	Optional 4–8 alphanumeric characters; use the actual intended transaction code

Illustrative synthetic request:

```
{
  "credential_configuration_id": "HarnessSdJwtCredential",
  "claims": {"fullName": "Synthetic Test Holder"},
  "expires_in": 600
}
```

Response subset:

```
{"credential_offer_uri":"<actual returned credential offer URI>"}
```

The returned URI can wrap a `credential_offer_uri` query parameter. The handoff extracts the actual offer-data URI, requires HTTPS, and passes it to the suite's live `credential_offer_endpoint`. It does not create a fake offer response.

5.3 Retrieve an offer

GET `/v1/certify/credential-offer-data/{offerId}` — JSON. Use the exact service-provided URI rather than assembling an offer ID yourself. The response contains the issuer, selected credential configuration and actual grant information. Codes are single-use sensitive values and are omitted here.

5.4 Native token exchange

POST `/v1/certify/oauth/token` — form URL-encoded request; JSON response. For native pre-authorization, the inspected controller reads:

```
grant_type=urn:ietf:params:oauth:grant-type:pre-authorized_code
pre-authorized_code=<actual unconsumed code>
tx_code=<actual transaction code, when required>
```

Response fields include `access_token`, `token_type` and expiry information. The retained native smoke flow returned a Bearer token. Tokens are not reproduced in this document. This smoke flow is not an issuer conformance pass.

The same native controller has an authorization-code branch for the native IAR flow; this does not establish conventional external-AS browser authorization, private_key_jwt client validation, or DPoP support. An external AS has its own code store. Pointing this request at Keycloak or eSignet cannot transfer native Certify codes automatically.

5.5 Credential issuance boundary

The credential endpoint is read from the **actual issuer metadata**, rather than invented by the handoff. The retained configuration routes issuance under `/v1/certify/issuance/credential`. The suite drives the selected issuance protocol after discovery and offer handling. Its payload requires real access authorization and signed holder proof; placeholders are not valid requests.

Successful complete issuance has not been established in the retained issuer conformance runs. Those runs were interrupted before the intended flow completed. Use tagged credential DTO/proof validators for the exact release contract rather than copying a draft-era proof body into the final-profile plan.

6. Verify APIs and DCQL presentation flow

6.1 Create a backend VP request

POST `/v1/verify/v2/vp-request` — expected HTTP 201, JSON.

Field	Harness-selected contract
`clientId`	Exact `redirect_uri:` prefix plus public direct-post URL
`transactionId`	Optional actual transaction correlation value
`nonce`	Optional; if supplied, at least 16 URL-safe ASCII characters
`dcqlQuery`	Required nonempty credential query
`responseCodeValidationRequired`	Must be false for this cross-device REST handoff

Illustrative request:

```
{
  "clientId": "redirect_uri:https://verify.test:8447/v1/verify/v2/vp-submission/direct-post",
  "dcqlQuery": {
    "credentials": [{
      "id": "pid",
      "format": "dc+sd-jwt",
      "meta": {"vct_values": ["urn:eudi:pid:1"]},
      "require_cryptographic_holder_binding": true
    }]
  },
  "responseCodeValidationRequired": false
}
```

The harness rejects legacy `presentationDefinition` and unsupported top-level fields. Query IDs must be unique and nonempty. Selected format must be `dc+sd-jwt` or `vc+sd-jwt`, with nonempty `vct_values` and holder binding required. Native DCQL validation remains authoritative for the full query.

Abbreviated response shape:

```
{
  "requestId": "<request-id>",
  "transactionId": "<transaction-id>",
  "authorizationDetails": {
    "clientId": "<actual requested client ID>",
    "responseType": "vp_token",
    "responseMode": "direct_post",
    "responseUri": "https://verify.test:8447/v1/verify/v2/vp-submission/direct-post",
    "nonce": "<service-generated nonce>",
    "dcqlQuery": {"credentials": ["<actual query objects>"]},
    "clientMetadata": {
      "vp_formats_supported": {
        "dc+sd-jwt": {
          "sd-jwt_alg_values": ["ES256"],
          "kb-jwt_alg_values": ["ES256"]
        }
      }
    }
  }
}
```

Placeholder strings in query arrays above abbreviate objects; they are not a valid replay payload. `clientMetadata` is published by the **reviewed derived native service**, not injected by Python. The original upstream DTO omitted it. The selected unsigned URL-query profile must not unexpectedly return a signed `requestUri`; the handoff fails if it does.

6.2 Relay actual authorization details to the suite

The harness sends a **GET to the suite-provided `submitUrl`**, with query fields:

Native response field	OID4VP parameter
`clientId`	`client_id`
`responseType`	`response_type`
`responseMode`	`response_mode`
`responseUri`	`response_uri`
`nonce`	`nonce`
`dcqlQuery`	JSON-encoded `dcql_query`
Native `clientMetadata`, if present	JSON-encoded `client_metadata`
Actual `requestId`	`state`

This is a suite interaction URL, not a new Verify endpoint. The handoff verifies the actual client ID, response binding and query before submitting. It does not invent metadata or alter cryptographic material.

6.3 Submit a presentation

POST `/v1/verify/v2/vp-submission/direct-post`, form URL-encoded. The suite acts as the presenting wallet in this flow.

Field	Meaning
`state`	Required actual request ID from the authorization request
`vp_token`	Actual serialized DCQL VP-token JSON containing signed presentations
`error`	Alternative wallet error response, when applicable
`error_description`	Optional accompanying error description

The selected success path requires a real VP token; missing or invalid submission parameters can be rejected. A 2xx response can mean the presentation was accepted for subsequent verification; it **does not prove the credentials passed**. Negative tests can legitimately receive 4xx before a VP is stored. Do not replace this call with a JSON file that claims success.

6.4 Poll request state

GET `/v1/verify/vp-request/{requestId}/status` — expected HTTP 200. Note that this route has **no `v2` segment**. Native state values consumed by the handoff are `ACTIVE`, `EXPIRED` and `VP_SUBMITTED`.

```
{"status":"VP_SUBMITTED"}
```

The native route can defer its response while waiting. An expired request is a real handoff failure. An unknown/invalid status is not interpreted as success.

6.5 Retrieve and verify results

POST `/v1/verify/v2/vp-results/{transactionId}` — expected HTTP 200, JSON.

Actual harness request:

```
{
  "skipStatusChecks": false,
  "statusCheckFilters": [],
  "includeClaims": false
}
```

Response schema subset:

```
{
  "transactionId": "<actual transaction-id>",
  "allChecksSuccessful": false,
  "credentialResults": ["<actual native credential result objects>"]
}
```

The harness requires the same transaction ID, a boolean overall outcome, and a nonempty list of actual result objects. Inspect native credential-result DTOs for their full per-check fields. The placeholder array is not a replay response. Unknown transaction IDs return 404 in the tagged controller. The actual result is saved as `<test-id>-verify-result.json` beside suite evidence.

A positive suite module fails if Verify rejects the presentation. A negative module is not made positive by changing native verification output. Backend success alone does not satisfy a pending screenshot requirement.

6.6 Browser-session APIs and unfinished UI integration

The tagged service also provides `POST /v1/verify/v2/vp-session-request`, returning HTTP 201 with real session-cookie management. Its UI SDK owns the create/poll/result lifecycle and uses corresponding session result APIs. This differs from the backend-owned REST request above.

The current harness does not yet implement genuine UI-owned transaction capture and screenshot upload tied to every affected suite test. Browser cookies cannot be replaced by an arbitrary backend request ID. The suite's screenshot/review requirement must be satisfied with the actual corresponding frontend result; JSON screenshots or a harness dashboard are not verifier UI evidence.

6.7 Standalone native credential verification and status resolution

The retained investigation also called the native `POST /v1/verify/v2/vc-verification` path to examine an actual signed credential after its status-list provider became unavailable. This is diagnostic evidence, not a replacement for the complete VP test plan. Supply the real signed credential using the selected release DTO and keep status checks enabled.

For the reviewed same-issuer-key JWT status-list profile:

- Valid credential/status: native status check succeeds.
- Revoked status: overall verification is false with `CREDENTIAL_REVOKED`.
- Unavailable status source: overall false with `STATUS_UNAVAILABLE`, even when the credential signature remains valid.
- Absence of optional credential status is not automatically an invalid credential.

The native checker fetches HTTPS `application/statuslist+jwt`, validates signature and freshness, decodes the referenced index and does not cache responses. Its supported profile does not imply support for every delegated signer or status format. Status error names are application results, not HTTP status codes.

7. Example use cases

7.1 Evaluate the API contract without a running stack

From the repository root in Windows PowerShell, using retained dependencies:

```
$env:PYTHONPATH = Join-Path $PWD '.local-deps'
python -m pytest runner/tests -q
```

This checks harness behavior, including mocked API-contract regressions and native tests when their runtime is available. It does not contact an unavailable service and does not establish protocol acceptance. See the README for a fresh install.

7.2 Trusted local health request

Illustrative Bash command, requiring an actual healthy deployment:

```
curl --cacert local/tls/ca.crt \
  -H 'Accept: application/json' \
  https://localhost:8447/v1/verify/actuator/health
```

Check the JSON body for `status=UP`. Do not use `-k`, publish private keys or claim this health check validates the presentation protocol.

7.3 Create a real Verify request

Write the synthetic request from section 6.1 to a local JSON file, then:

```
curl --cacert local/tls/ca.crt \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json' \
  --data-binary @local/verify-request.json \
  https://localhost:8447/v1/verify/v2/vp-request
```

Expect 201 only if the actual service validates and creates the request. Retain the actual correlation IDs privately; then let the suite submit its signed presentation and query the result. These commands were not executed for this PDF.

7.4 Run full diagnostic workflows

After the proper stack and trust configuration are healthy, use Git Bash:

```
bash run-full-stack-conformance.sh --component certify --no-up
bash run-full-stack-conformance.sh --component verify --no-up
bash run-full-stack-conformance.sh --combined --no-up
```

Each wrapper uses the suite REST API and component handoffs described above. Do not treat a short diagnostic subset as a full plan. Use the existing project's Compose files/environment rather than accidentally creating a competing stack.

7.5 Retrieve results using the Python HTTP contract

Illustrative trusted request; retrieve identifiers from an actual transaction:

```
import requests

transaction_id = "<actual transaction id>"
response = requests.post(
    "https://localhost:8447/v1/verify/v2/vp-results/" + transaction_id,
    json={"skipStatusChecks": False,
          "statusCheckFilters": [], "includeClaims": False},
    verify="local/tls/ca.crt",
    allow_redirects=False,
    timeout=30,
)
response.raise_for_status()
result = response.json()
# Evaluate the actual boolean and credential checks; do not print sensitive claims.
```

Use URL-safe/encoded actual IDs in real callers. Do not retry ambiguous writes merely because the response was lost. The harness's own client applies additional deadlines and response-shape checks.

8. Timeouts, retries and error handling

Condition	Harness behavior
Read connection failure or timeout	Up to three attempts, within the actual deadline
Ambiguous lost response to a write	Fail; no automatic replay of a possibly completed write
State-changing suite callback GET	Retry disabled despite GET method
Unexpected HTTP status	Sanitized API error; do not echo secret-bearing response body
Invalid JSON/shape	Infrastructure error, not a skipped module
Invalid/empty ZIP export	Evidence failure and infrastructure exit
Module/global timeout	Preserve partial results; cancel abandoned runner where possible
Screenshot WAITING timeout	FAILED, even when backend verification succeeded
Wrong configured suite origin	Refuse request to prevent credential/TLS setting leakage

HTTP requests have a 30-second cap within module/global budgets. Native status resolution has its own bounded network and payload policies. The wrapper's startup/health budgets are separate from protocol/API execution time.

Typical error categories, without claiming every upstream body is identical:

- 400: invalid query/body/DCQL/submission or OAuth parameters, depending on service.
- 401/403: authentication, authorization or CSRF policy failure, depending on route.
- 404: unknown plan/request/transaction or unavailable route.
- 5xx: service/runtime failure; inspect redacted service diagnostics.
- TLS/connection failure: missing CA, hostname mismatch, unavailable gateway or engine.

Always inspect the selected native controller/error DTO privately for exact response codes and fields. Do not normalize all failures into a synthetic success.

9. Local output contract and build exits

The following is a **local result file**, not an HTTP endpoint:

```
{
  "component": "verify",
  "runId": "<unique-run-id>",
  "planId": "<actual-plan-id>",
  "modules": [{
    "name": "<actual-module-name>",
    "testId": "<actual-test-id>",
    "variant": {},
    "status": "FAILED",
    "detail": "<sanitized actual reason>",
    "logUrl": "<actual suite log URL>",
    "duration": 1.0
  }],
  "summary": {"passed": 0, "failed": 1},
  "errors": [],
  "exitCode": 1
}
```

This abbreviated illustrative example is not a retained test result. Full files also carry timestamps, duration, selected module information, benchmark evaluation and evidence paths. Actual counts must come from the relevant invocation.

Exit	Meaning
0	No unexpected failure gate; does not make REVIEW/SKIPPED/WARNING into passes
1	Protocol FAILED/INTERRUPTED or configured execution-budget breach
2	Invalid configuration, unavailable API or infrastructure/evidence failure

Statuses include PASSED, FAILED, INTERRUPTED, WARNING, SKIPPED, REVIEW, EXPECTED_FAILURE and EXPECTED_SKIP. Unknown results fail. Infrastructure failures cannot be allowlisted. Java adapters preserve failing process/result states and add an independent evaluated benchmark row when applicable.

`configs/benchmark-thresholds.json` uses null defaults. Null means `NOT_CONFIGURED`, not performance PASSED. Configured thresholds use inclusive boundaries: measured seconds equal to the limit pass, greater values fail. Combined execution sums sequential component execution time and preserves failed component gates. No arbitrary production threshold is invented.

Evidence locations:

- `results/results-certify.json`, `results/results-verify.json` and component Markdown.
- `results/results-combined.json` and `results/report-combined.md`.
- `results/<component>-evidence/<plan-id>/`, containing logs, exports and native results.
- Standalone TestNG/Extent output under the configured Maven report directory.

- `artifacts/demo/final-submission-readiness.md` for latest assessment and current checks.

10. Known limitations and current evidence

The fresh 2026-11-02 regression assessment recorded **90 passed, 0 failures, 0 errors and 0 skips**, with one Python deprecation warning. Static Compose validation and Java adapter compilation passed. Docker's Linux engine was unavailable, so no fresh service/API/conformance run is implied by this PDF.

Retained 2026-10-31 results, explicitly historical:

Workflow	Passed	Failed	Interrupted	Warning	Skipped	Exit
Certify	0	0	12	1	1	1
Verify	5	7	0	0	0	1
Combined	5	7	12	1	1	1

Remaining blockers are real external-AS/client/grant integration, Certify DPoP resource validation and genuine Verify UI evidence/review. Published native metadata does not by itself resolve authorization compatibility. Keycloak and eSignet are assessed candidates, not deployed integrations.

The upstream suite's protocol HTTP client has its own permissive TLS behavior. Its protocol result does not prove TLS trust; independent normal-JDK/Python checks are separate evidence. See `docs/tls-validation.md`.

11. Source and documentation index

Consult these repository files when evaluating or changing the contract:

Area	Source / reference
Complete setup and workflow commands	`README.md`
Suite API implementation	`runner/conformance_runner.py` and its regression tests
Component API handoffs	`runner/handoff_certify.py`, `runner/handoff_verify.py`
API client security/retry behavior	`runner/common.py`, `SuiteAPI` in the runner
Certify native routes	Tagged `PreAuthorizedCodeController`, `OAuthController`, `WellKnownController`
Certify request DTO	Tagged `certify-core/.../PreAuthorizedRequest.java`
Verify native routes	Tagged `VPRequestController`, `VPSubmissionController`, `VPResultController`
Verify DTOs	Tagged `VPRequestCreateDto`, `VPVerificationResultDto`, `VerificationRequestDto`
Derived native changes	`patches/`, `tools/verify-fixes/`, `tools/prepare_native_images.py`
Actual handoff findings	`docs/component-handoff.md`, `docs/blocker-resolution.md`
External-AS prerequisites	`docs/external-as-assessment.md`
Execution-time policies	`docs/benchmark-thresholds.md`
Troubleshooting and TLS	`docs/troubleshooting.md`, `docs/tls-validation.md`
Historical full validation	`artifacts/demo/final-validation-report.md`

Documentation examples omit secrets and intentionally do not provide fake signed credentials, fake discovery metadata or a fictitious successful external-AS flow. Use actual service responses and approved test identities when the deployment is available.

> Generated from docs/api-reference.md. Examples contain no real credentials. This document is not an end-to-end conformance certificate.